

Infografía

Módulo 6 — Transformaciones 2D / 3D

UPB · Gestión I/2026

Plan de la clase

1. **¿Por qué transformaciones?** — el ladrillo que ya estábamos usando sin nombrarlo.
2. **A mano** — traslación y rotación con fórmulas vértice por vértice.
3. **Coordenadas homogéneas** — $(x, y) \rightarrow (x, y, 1)$ y matrices 3×3 .
4. **Composición** — el orden importa: $A \cdot B \neq B \cdot A$.
5. **Caso interactivo** — tanque que rota y avanza a 60 FPS.
6. **Salto a 3D** — matrices 4×4 , tres rotaciones, proyección.
7. **Tarea** — sistema solar (jerarquía) + visor 3D con perspectiva.

*Hasta ahora pymunk movía bodies por nosotros; arcade rotaba sprites. Hoy **abrimos esa caja** y vemos el motor de movimiento por dentro.*

1. ¿Por qué?

Lo que ya hicimos sin nombrar

- Módulo 3 (rebote): mover `(x, y)` cada frame = **translación**.
- Módulo 4 (sprite con `sprite.angle`): rotar la textura = **rotación**.
- Módulo 5 (`body.angle → sprite.radians`): pymunk te daba un ángulo, arcade lo aplicaba al sprite = **rotación + traslación, escondidas**.

Todo movimiento de un objeto rígido se descompone en: trasladar, rotar y (a veces) escalar.

Hoy vamos a **escribirlas a mano** para entender qué hace el motor por debajo. Es el mismo patrón didáctico del curso: usamos la herramienta, después la abrimos.

Tres operaciones, un único objeto

Para cualquier polígono (lista de vértices) podemos:

Operación	Qué cambia
Translación	desplaza todo el polígono en (dx, dy)
Escala	agranda o achica respecto a un pivote
Rotación	gira un ángulo θ alrededor de un pivote

No hace falta nada más para mover, animar, jerarquizar y proyectar objetos en 2D y 3D. Toda la geometría interactiva del curso vive sobre estos tres ladrillos.

2. A mano

01_naive_transform.py

Traducción: fórmula directa

$$(x, y) \rightarrow (x + d_x, y + d_y)$$

```
def translate(self, dx, dy):  
    self.vertices = [(x + dx, y + dy) for x, y in self.vertices]
```

Aplico la misma operación a cada vértice. **No hay matriz**: una list comprehension y listo.

Rotación alrededor del origen

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

```
xr = x * cos_t - y * sin_t  
yr = x * sin_t + y * cos_t
```

¡Pero esto rota alrededor de **(0, 0)**! Si el cuadrado vive en (100, 100), gira haciendo un arco gigante alrededor del origen — no sobre su propio centro.

Rotación alrededor de un pivote

Truco: trasladar el pivote al origen, rotar ahí, volver.

```
def rotate(self, theta, px, py):  
    for x, y in self.vertices:  
        x0, y0 = x - px, y - py          # 1. al origen  
        xr = x0*cos_t - y0*sin_t         # 2. rotar  
        yr = x0*sin_t + y0*cos_t  
        result.append((xr + px, yr + py)) # 3. devolver
```

tres pasos: trasladar -p, rotar en origen, trasladar +p

Esto va a aparecer todo el módulo. Es la receta de "operar respecto de un pivote".

Demo: 01_naive_transform.py

```
cd 6._transformations  
uv run 01_naive_transform.py
```

- Rojo: cuadrado original.
- Verde: cuadrado rotado **-75° alrededor de (100, 100)**.
- Cyan: cuadrado trasladado.
- Punto amarillo: el pivote.

*Funciona, pero cada operación es código nuevo (las fórmulas mezcladas con la lógica del polígono). Si quiero combinar varias, se vuelve un fideo. La solución es **cambiar de lenguaje**.*

3. Coordenadas homogéneas

el truco que lo cambia todo

El problema

La translación es $(x, y) \rightarrow (x + d_x, y + d_y)$ – una **suma**.

La rotación / escala son **multiplicaciones** por una matriz 2×2 .

¿Cómo expreso las dos como **la misma operación**? Necesito un único tipo de objeto que las represente.

Idea: subir una dimensión. Pasar de (x, y) a $(x, y, 1)$ y usar matrices 3×3 .

Las tres matrices

$$T(d_x, d_y) = \begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix} \quad S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Cada vértice ahora vive como columna $\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$, y aplicar una transformación es **un solo producto matriz × vector**.

Aplicar una matriz a todo el polígono

```
def apply(self, M):  
    V = np.array([[x, y, 1] for x, y in self.vertices]).T    # 3 x N  
    V2 = M @ V                                              # 3 x N  
    self.vertices = [(V2[0, i], V2[1, i]) for i in range(V2.shape[1])]
```

- `V` apila todos los vértices como columnas.
- `M @ V` aplica `M` a cada vértice en una sola multiplicación de matrices (numpy lo vectoriza).
- Descarto la fila homogénea (que sigue siendo `1`) y vuelvo a tuplas.

Una función ``apply``. Las matrices T, S, R son solo "qué le paso".

Rotación alrededor de pivote: ahora como producto

$$M_{rot_pivot} = T(p_x, p_y) \cdot R(\theta) \cdot T(-p_x, -p_y)$$

```
def rotate(self, theta_deg, pivot):  
    px, py = pivot  
    self.apply(T(px, py) @ R(theta_deg) @ T(-px, -py))
```

*Comparar con la versión a mano: la receta es **la misma** (trasladar, rotar, trasladar), pero ahora son tres matrices multiplicadas, no nueve líneas de aritmética sobre cada vértice.*

Regla del orden: derecha a izquierda

$$M = A \cdot B \quad \Rightarrow \quad M \cdot v = A \cdot (B \cdot v)$$

Lo **primero** que se aplica al vector es lo de la **derecha**.

```
# leer: "primero R, luego T"  
M = T(px, py) @ R(theta) @ T(-px, -py)  
#           ^ se aplica   ^ esto va primero
```

En la mayoría de la literatura de gráficos esto está al revés (vectores fila vs columna). Acá: **vectores columna, derecha a izquierda**. Mantener una convención y no mezclar.

Demo: 02_matrix_transform.py

```
uv run 02_matrix_transform.py
```

El resultado visual es idéntico a 01 — misma escena, mismo cuadrado verde rotado, mismo cuadrado cyan trasladado.

Lo que cambió: el **código** ahora es matrices, no fórmulas. Una vez aceptado el lenguaje, agregar operaciones es trivial: aparecen `scale()` y `rotate(pivot)` en pocas líneas.

Misma física, distinto lenguaje. El cambio paga al combinar operaciones.

4. Composición

el orden importa

$$A \cdot B \neq B \cdot A$$

Las matrices **no conmutan** (en general). Tomamos los mismos números, los aplicamos en distinto orden, **el resultado es distinto**.

Camino A (verde)

$$M_a = T(250, 0) \cdot R(45^\circ)$$

"rotar y después trasladar"

Camino B (magenta)

$$M_b = R(45^\circ) \cdot T(250, 0)$$

"trasladar y después rotar"

B traslada el cuadrado y después lo "barre" por un arco grande alrededor del origen.

Demo: 03_compose_order.py

```
uv run 03_compose_order.py
```

- Gris: original.
- Verde: $T \cdot R \rightarrow$ primero rota en su lugar, después se aleja.
- Magenta: $R \cdot T \rightarrow$ primero se aleja, después la rotación lo manda por un arco.

*La intuición útil: **$R(\theta)$ rota alrededor del origen**. Si trasladaste al cuadrado antes, esa rotación lo hace orbitar. Si rotaste primero, la rotación queda local y la translación la lleva a su lugar.*

Caso útil: pivote arbitrario

Querés rotar un cuadrado **alrededor de su centro**, pero está lejos del origen.

$$M = T(c_x, c_y) \cdot R(\theta) \cdot T(-c_x, -c_y)$$

1. Mover el centro al origen.
2. Rotar.
3. Devolver el centro a su lugar.

Esto es exactamente lo que hace `rotate(theta, pivot=...)` en el ejemplo 02. Y lo van a ver de nuevo en el sistema solar (ejercicio 0): el planeta orbita "alrededor del sol" usando el mismo truco.

5. Caso interactivo

04_tank_game.py

Composición acumulativa por frame

Cada frame:

- $\Delta\theta = \omega \cdot dt$ – un poquito más de rotación.
- $\Delta x = v \cos \theta \cdot dt$, $\Delta y = v \sin \theta \cdot dt$ – un poquito más de avance.

```
self.body.rotate(dtheta, pivot=(self.x, self.y))  
self.body.translate(dx, dy)
```

No recalcular desde cero. Acumular sobre la pose actual.

*Esto es **exactamente** lo que hacía pymunk en el módulo 5 – pero ahora estamos del otro lado del mostrador: somos nosotros los que aplicamos los deltas a las matrices.*

Demo: 04_tank_game.py

```
uv run 04_tank_game.py
```

Controles:

- **flechas arriba/abajo** → velocidad lineal.
- **flechas izq/der** → velocidad angular.
- **click izquierdo** → disparar bala (un punto rojo que viaja en línea recta).
- **r** → respawn.

Cosas para mirar:

- Las balas **no rotan con el tanque** después de salir. Se les fija el ángulo en el momento del disparo.
- El tanque rota **sobre su propio centro** (pivote = posición actual).

6. 3D

el mismo cuento, una dimensión más

¿Qué cambia al saltar a 3D?

Casi nada conceptualmente:

2D	3D
Vértice $(x, y, 1)$	Vértice $(x, y, z, 1)$
Matrices 3×3	Matrices 4×4
Una rotación: $R(\theta)$	Tres rotaciones: R_x, R_y, R_z
Dibujo directo	Hay que proyectar $3D \rightarrow 2D$

El patrón "T, S, R en homogéneas, componer con @" se mantiene. Solo que ahora T tiene tres componentes y hay tres rotaciones distintas.

Las tres rotaciones

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Cada una deja un eje **fijo** y rota los otros dos. Y `R_x @ R_y != R_y @ R_x` — recordar el orden.

Proyección 3D → 2D

Necesitamos llevar los vértices 3D a píxeles de pantalla. Dos opciones simples:

Ortográfica (la que vemos hoy): tirar la **z**.

$$(x, y, z) \rightarrow (x + \frac{W}{2}, y + \frac{H}{2})$$

Perspectiva (en el ejercicio 1): lejos = más chico.

$$(x, y, z) \rightarrow (x \cdot \frac{D}{D+z} + \frac{W}{2}, y \cdot \frac{D}{D+z} + \frac{H}{2})$$

donde D es una "distancia focal" (cuán lejos está la cámara).

Demo: 05_3d_wireframe.py

```
uv run 05_3d_wireframe.py
```

- Cubo en wireframe (lista de **vértices** + lista de **aristas**).
- Rota constantemente alrededor de **y** con un toque de inclinación en **x**.
- Proyección ortográfica: las caras de atrás se ven del mismo tamaño que las del frente.

Sin perspectiva, mirando de frente, el cubo se ve como un cuadrado con líneas adentro. Hay que esperar a la rotación para que se "vea" 3D. Por eso en el ejercicio 1 agregamos perspectiva.

Patrón general

Lo que se repite en todos los ejemplos

```
# 1. construir matrices basicas
T(...), S(...), R(...)    # 3x3 en 2D, 4x4 en 3D

# 2. componer con @ (derecha a izquierda)
M = T(cx, cy) @ R(theta) @ T(-cx, -cy)

# 3. aplicar al objeto (lista de vertices)
self.vertices = (M @ V_homogenea)[: -1, :].T
```

Sumado a:

- **Rotación con pivote** = $T(p) @ R @ T(-p)$.
- **Composición frame a frame** = acumular deltas sobre la pose actual.
- **Jerarquía** = $M_{\text{hijo}} = M_{\text{padre}} @ M_{\text{local}}$.

Cualquier motor de juegos (Godot, Unity, Three.js) por dentro es esto, repetido a otra escala.

Resumen

- Traducción, escala, rotación → **una sola receta**: matriz en coordenadas homogéneas.
- **Componer = multiplicar matrices**. El orden importa.
- Rotar alrededor de un pivote = $T \cdot R \cdot T^{-1}$.
- Animar = acumular deltas sobre la pose anterior.
- 3D = lo mismo, pero con $(x, y, z, 1)$ y matrices 4×4. Más: hay que **proyectar**.

Hoy escribimos el motor de transformaciones que pymunk y arcade usaban escondido. Saben para qué sirve cada operación.

Ejercicios

Ejercicio 0 — sistema solar 2D

`ex0_solar_system.py` — transformaciones jerárquicas.

Sol en el centro, planeta orbitando al sol, luna orbitando al planeta. Cada uno gira sobre sí mismo además de orbitar.

4 TODOs que construyen una jerarquía:

$$M_{moon} = M_{planeta_centro} \cdot R(\theta_{moon}) \cdot T(r_{moon}, 0) \cdot R(spin_{moon})$$

Mismo patrón que usa cualquier scene graph (Godot Node hierarchy, Unity transform.parent, Three.js Object3D.add). La luna hereda la pose del centro del planeta.

Ejercicio 1 — visor 3D con perspectiva

`ex1_3d_viewer.py` — extender `05` con:

- **Perspectiva** real: $factor = D / (D + z)$.
- **Controles de teclado** para rotar la escena en X, Y, Z.
- **Múltiples objetos** (cubo + pirámide).

4 TODOs. La solución completa queda como referencia en `ex1_3d_viewer_solved.py`.

¿Preguntas?

Próxima clase: Godot 4 — los conceptos del módulo aplicados en un motor real (Node, Transform, escena 3D).

`uv run` todos los ejemplos durante la semana — leer comentarios.